

COEN383
Advanced Operating System
Project 3 Report - Ticket Seller Simulation
Group No. #1

(Gaurav Gaikwad, Gauri Dave, Wineel Wilson Dasari, Anthony Bryson, Dhanashree Bharuka)

Introduction

Using ten concurrent threads to simulate ticket sellers this project duplicates a realistic ticket-selling environment. Through simulation the sellers work continuously for an hour while serving waiting customers. The system implements OS-level principles through multithreading together with thread synchronization and resource sharing and scheduling techniques which operate through Pthreads library and C programming language programming constructs.

System Design and Implementation

Seller Configuration

- **1 High-Priced Seller (H):** Assigns seats from the front row (row 0) to the back.
- **3 Medium-Priced Sellers (M):** Start at the middle row (row 5) and alternate outward (5 → 6 → 4 → 7...).
- **6 Low-Priced Sellers (L):** Assign seats from the back row (row 9) to the front.

Service Time

- **H sellers:** 1-2 minutes
- **M sellers:** 2-4 minutes
- **L sellers:** 4-7 minutes

Each seller has a queue of customers who arrive at random times between 0 and 59. The queues are sorted by arrival time and processed accordingly.

Modules and File Structure

File Name	Usage
<i>main.c</i>	Controls the simulation clock and coordinates all sellers
<i>seller.c / seller.h</i>	Handles seller thread logic, service timing, and queue processing

<i>queue.c / queue.h</i>	Manages customer queue operations and sorting
<i>seatmap.c / seatmap.h</i>	Controls seat assignment and final seat chart printing
<i>Makefile</i>	Automates compilation
<i>README.md</i>	Project documentation and instructions
<i>output/-</i>	Saved output logs (Output1.txt, Output2.txt, Output3.txt)

Clock and Thread Synchronization

A shared simulation clock advances from minute 0 to 59.

- The main thread uses *sleep(1)* or *usleep(100000)* per minute.
- Seller threads wait using *pthread_cond_wait()* and are awakened by *pthread_cond_broadcast()*.

Shared data such as the current time, seat matrix, and thread counters are protected with mutexes.

A *simulation_ended* flag and *active_thread* counter ensure all threads exit properly.

Seat Assignment Logic

Seat assignment strictly follows seller types:

- H sellers allocate from row 0 to 9 (left to right)
- M sellers start at row 5 and alternate outward
- L sellers allocate from row 9 to 0 (right to left)

All seat updates are thread-safe using a global mutex to avoid race conditions.

Simulation Control and Speed Toggle

The simulation can run in:

- **Normal mode (*sleep(1)*):** 1 real second = 1 simulated minute
- **Fast mode (*usleep(100000)*):** 0.1 second = 1 simulated minute (enabled by *FAST_MODE = 1* in *main.c*)

Compilation and Execution

Using Make

```
cd src
make
./main 5
```

Saving Output

```
./main 5 > ../output/Output1.txt
./main 10 > ../output/Output2.txt
./main 15 > ../output/Output3.txt
```

Cleaning Build

```
make clean
```

Sample Output

Each run outputs:

- Logs of arrivals, service, and seat assignments
- Final seating chart (10x10)
- Summary statistics:
 - Customers served and returned per seller type
 - Placeholder values for Average TAT, RT, and Throughput

Challenges and Learnings

Thread synchronization:

Properly coordinating thread execution with a global clock was challenging. Fixes included using *simulation_ended* and broadcasting one final wakeup.

Deadlocks:

Early implementations caused hanging threads. These were fixed by ensuring condition variables are always signaled after the simulation loop.

Thread-safe seat allocation:

Implementing thread-safe logic for a shared seat matrix required mutex-protected critical sections.

Conclusion:

The system implements a realistic simulation of a multithreaded ticketing system that includes several types of sellers. The project implementation served to reinforce complex operating system concepts including thread lifecycle handling as well as synchronization techniques and shared resource management methods and concurrency mechanisms. The project remains robust and accurate for future use because of its fast-mode toggle and clean modular structure and correct synchronization features.